

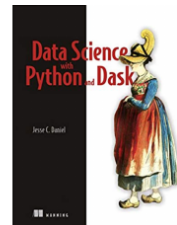
09.3: Python Dask



UMD DATA605 - Big Data Systems

09.3: Python Dask

- **Instructor:** Dr. GP Saggese, gsaggese@umd.edu
- @Resources@
 - Web resources:
 - Dask project
 - Dask examples
 - Tutorial
 - Dask_tutorial
 - Dask_advanced_tutorial
 - Class project
 - Mastery
 - Data science with Python and Dask, 2019



1 / 16

2 / 16: Dataset Size Issues

Dataset Size Issues

- **Small datasets (< 1 GB)**
 - Fits into RAM
 - No disk paging needed
- **Medium dataset (< 1TB)**
 - Doesn't fit into RAM
 - Fits into local disk
 - Performance penalty with local disk
 - Need multiple CPU cores
 - Difficult to leverage parallelism with Python/Pandas
- **Large dataset (> 1TB)**
 - Doesn't fit into RAM
 - Doesn't fit into local disk
 - Need multiple servers
 - Python/Pandas not built for distributed datasets
 - Use frameworks for massive datasets
 - E.g., Hadoop, Spark, Dask, Ray



2 / 16

- **Small datasets (< 1 GB)**
 - These datasets can easily fit into a computer's RAM, allowing for fast data processing without the need for disk paging. This means that operations can be performed quickly and efficiently using tools like Python and Pandas.
- **Medium datasets (< 1TB)**
 - Medium-sized datasets are too large to fit into RAM but can be stored on a local disk. However, accessing data from a disk is slower than from RAM, leading to a performance penalty.
 - To handle these datasets effectively, multiple CPU cores are needed. However, leveraging parallelism in Python, especially with Pandas, can be challenging due to its limitations in handling parallel processing.
- **Large datasets (> 1TB)**
 - These datasets exceed the capacity of both RAM and local disk storage, requiring multiple servers for processing.
 - Python and Pandas are not designed for distributed datasets, so alternative frameworks are necessary. Tools like Hadoop, Spark, Dask, and Ray are specifically built to manage and process massive datasets across distributed systems, enabling efficient data handling and analysis.

3 / 16: Dataset Size Issues

Dataset Size Issues

@Category@	@Size@
Small datasets	< 1 GB
Medium datasets	< 1 TB
Large datasets	> 1 TB

- **The thresholds are fuzzy and changing over time**
 - Scale computer 10x to get 10x bigger datasets
- **Problem with scaling datasets**
 - Long run times
 - Rewriting code for different dataset sizes
 - Plan what and how to do efficiently
 - Cumbersome framework (Pandas easy, Hadoop difficult)

3 / 16

- **Dataset Size Issues**
- **Small datasets:** These are datasets that are less than 1 gigabyte in size. They are typically easy to handle with standard tools like Excel or simple scripts in Python or R.
- **Medium datasets:** These datasets are less than 1 terabyte. They require more robust tools and computing power, such as SQL databases or more advanced data processing libraries like Pandas.
- **Large datasets:** These datasets exceed 1 terabyte and often require distributed computing frameworks like Hadoop or Spark to process efficiently.
- **The thresholds are fuzzy and changing over time**
 - As technology advances, the definitions of what constitutes a small, medium, or large dataset can change. What was considered large a few years ago might now be seen as medium due to improvements in storage and processing capabilities.
 - *Scaling up* computing resources by 10 times can allow you to handle datasets that are 10 times larger, but this isn't always straightforward or cost-effective.
- **Problem with scaling datasets**
 - **Long run times:** As datasets grow, the time required to process them increases, which can be a bottleneck in data analysis.
 - **Rewriting code for different dataset sizes:** Code that works for small datasets might not be efficient or even feasible for larger datasets, necessitating rewrites or optimizations.

- **Plan what and how to do efficiently:** It's crucial to plan your data processing strategy to ensure efficiency, especially as datasets grow in size.
- **Cumbersome framework:** Some frameworks are easier to use than others. For example, Pandas is user-friendly for small to medium datasets, while Hadoop can be more complex and challenging to work with for large datasets.

4 / 16: Dask

Dask

- **Dask is written in Python**
 - Scales Numpy, Pandas, sklearn
 - Dask objects wrap library objects (e.g., Pandas DataFrame, numpy array)
 - Parallel parts are “chunks” or “partitions”
 - Queued for work
 - Shipped between machines
 - Worked locally
- **Pros**
 - Use familiar interfaces
 - Write code optimized for parallelism
 - Dask handles heavy lifting
- **Scaling Dask is easy**
 - Prototype on local machine, use cluster when needed
 - No code refactoring needed
 - No cluster-specific issues
 - E.g., resource management, data recovery, data movement



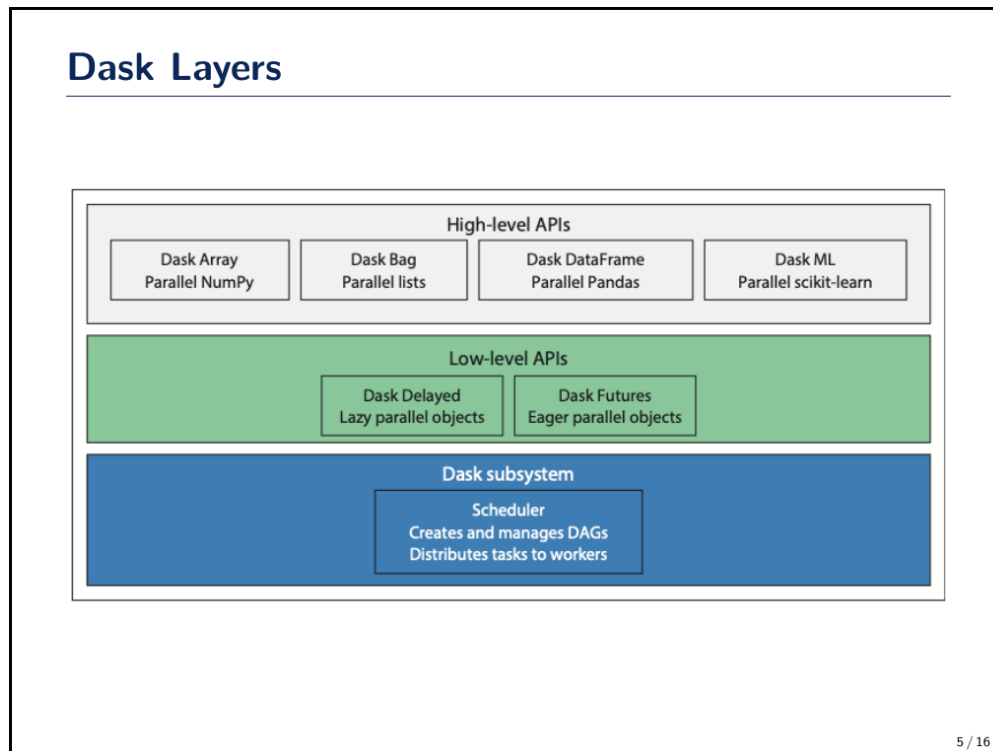
The Dask logo consists of three overlapping squares in yellow, orange, and red, followed by the word "dask" in a bold, dark blue, lowercase sans-serif font.

4 / 16

- **Dask is written in Python**
 - Dask is designed to scale popular Python libraries like Numpy, Pandas, and sklearn. This means you can work with larger datasets and more complex computations without changing your existing codebase significantly.
 - Dask objects act as wrappers around library objects such as Pandas DataFrames or numpy arrays. This allows you to use familiar tools while benefiting from Dask's parallel computing capabilities.
 - The parallel processing in Dask is achieved through “chunks” or “partitions.” These are smaller pieces of your data that can be processed independently.
 - These chunks are queued for work, allowing efficient task scheduling.
 - They can be distributed across different machines, enabling scalable computations.
 - Local processing is also possible, making it versatile for different environments.
- **Pros**
 - Dask allows you to use interfaces you are already familiar with, reducing the learning curve.
 - You can write code that is optimized for parallel execution, and Dask takes care of the complex parts of parallel computing.
- **Scaling Dask is easy**

- You can start by prototyping on your local machine and then scale up to a cluster when necessary, without needing to rewrite your code.
- Dask abstracts away cluster-specific issues like resource management and data recovery, simplifying the scaling process.
- It can run on multi-core systems and integrates with various cluster managers such as Yarn, Mesos, Kubernetes, and AWS ECS, providing flexibility in deployment.

5 / 16: Dask Layers



- **High-level APIs**
 - *Dask Array*: This is similar to NumPy but designed for parallel computing. It allows you to work with large arrays that don't fit into memory.
 - *Dask Bag*: Functions like parallel lists, useful for processing semi-structured data like JSON or log files.
 - *Dask DataFrame*: Mimics Pandas DataFrames but operates in parallel, enabling handling of larger-than-memory datasets.
 - *Dask ML*: Integrates with scikit-learn to provide parallel machine learning capabilities.
- **Low-level APIs**
 - *Dask Delayed*: Allows for lazy evaluation, meaning computations are only performed when needed. This is useful for building complex workflows.
 - *Dask Futures*: Provides eager execution, where tasks are executed immediately. This is beneficial for real-time data processing.
- **Dask Subsystem**
 - *Scheduler*: The core component that manages task execution. It creates and manages Directed Acyclic Graphs (DAGs) to organize tasks and efficiently distribute them across workers. This ensures optimal resource utilization and parallel processing.

This slide illustrates how Dask is structured to handle large-scale data processing by providing both high-level abstractions for ease of use and low-level controls for flexibility.

6 / 16: Scaling Up vs Scaling Out

Scaling Up vs Scaling Out

- **Scaling up**
 - Replace equipment with larger, faster options
 - E.g., buy a larger pot, replace knife with food processor
 - **Pros**
 - Better hardware, no code changes needed
 - **Cons**
 - Exceed current machine capacity eventually
 - Cost: more powerful machines are expensive
- **Scaling out**
 - Divide work between many workers in parallel
 - E.g., buy more pots and hire more cooks
 - **Pros**
 - Task scheduler organizes computation, assigns workers to tasks




6 / 16

- **Scaling Up**
 - This approach involves upgrading existing equipment to more powerful versions. For example, replacing a small pot with a larger one or using a food processor instead of a knife.
 - **Pros**
 - You get better hardware performance without needing to change your existing code.
 - **Cons**
 - Eventually, you might hit the limits of what a single machine can handle.
 - More powerful machines can be quite expensive, increasing costs significantly.
- **Scaling Out**
 - This method involves distributing tasks across multiple workers or machines, similar to hiring more cooks and using more pots in a kitchen.
 - **Pros**
 - A task scheduler can efficiently manage and assign tasks, making it a cost-effective solution.
 - You don't need specialized hardware, which can save money.
 - **Cons**
 - You need to write code that can handle parallel processing, which can be complex.
 - Managing and maintaining a cluster of machines can incur additional costs.

7 / 16: Dask: Computation

Dask: Computation

- **Lazy computations**
 - Define transformations on data
 - Define next computation without waiting
 - Operate in chunks to avoid loading entire data in memory
 - E.g.,
 - Split 2GB file into 32 64MB chunks
 - Operate on 8 chunks per server
 - Max memory use: 512MB = (8 × 64MB)
 - Track object dimensions and data types
 - No code execution
- `compute()`
 - Run computation (materialize)
 - `missing_count_pct = missing_count.compute()`
- `persist()`
 - Discard intermediate work to minimize memory
 - Re-run graph for additional computation on intermediate nodes
 - Keep intermediate result in memory
 - Speed up large, complex DAGs for reuse

7 / 16

- **Dask: Computation**- **Lazy computations**

- *Lazy computations* mean that Dask doesn't immediately execute operations when you define them. Instead, it sets up a plan or a series of transformations that will be applied to the data later. This allows you to define what you want to do without actually doing it right away.
- By defining the next steps without waiting for the current ones to finish, Dask can optimize the entire process, making it more efficient.
- Dask processes data in chunks, which is crucial for handling large datasets that don't fit into memory all at once. For example, if you have a 2GB file, you can split it into smaller 64MB chunks. This way, you can process these chunks one at a time or in parallel, using less memory.
- In the example, operating on 8 chunks per server means you only need 512MB of memory at a time, which is much more manageable.
- Dask keeps track of the size and type of data objects, but it doesn't execute any code until you tell it to.

- `compute()`

- The `compute()` function is used to actually run the computations you've defined. This is when Dask takes all the planned operations and executes them to produce a result.
- For instance, if you have a variable `missing_count` that represents a series of operations to count missing values, calling `missing_count.compute()` will perform those operations and give you the final count.

- `persist()`
 - The `persist()` function helps manage memory by discarding unnecessary intermediate results. This is useful when you want to keep certain results in memory for further use without redoing all the previous computations.
 - It allows you to keep intermediate results in memory, which can be reused for additional computations, speeding up the process, especially for complex tasks that involve large Directed Acyclic Graphs (DAGs).

8 / 16: Dask: Data Structures

Dask: Data Structures

- **Dask DataFrame**
 - Implements Pandas DataFrame
 - Tabular/relational data
- **Dask Array**
 - Implements numpy ndarray
 - Multidimensional array
- **Dask Bag**
 - Coordinates Python lists of objects
 - Parallelize computations on unstructured/semi-structured data

- **Dask DataFrame**
 - *Implements Pandas DataFrame*: Dask DataFrame is designed to work like a Pandas DataFrame but can handle larger-than-memory datasets by breaking them into smaller, manageable chunks.
 - *Tabular/relational data*: It is ideal for working with structured data, similar to what you would find in a database or spreadsheet, allowing for operations like filtering, grouping, and aggregating.
- **Dask Array**
 - *Implements numpy ndarray*: Dask Array mimics the functionality of NumPy arrays, enabling operations on large, multi-dimensional arrays that don't fit into memory.
 - *Multidimensional array*: This structure is useful for scientific computing and data analysis tasks that require handling large datasets with multiple dimensions.
- **Dask Bag**
 - *Coordinates Python lists of objects*: Dask Bag is designed for processing collections of Python objects, similar to a list, but optimized for parallel computation.
 - *Parallelize computations on unstructured/semi-structured data*: It is particularly useful

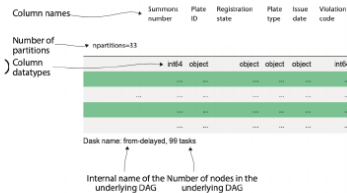
for tasks involving text processing or JSON data, where the data structure is not strictly tabular.

9 / 16: Dask Reading Data

Dask Reading Data

– Consider:

```
import dask.dataframe as dd
df = dd.read_csv('nyc-parking-tickets-2017.csv')
missing_values = df.isnull().sum()
missing_values
```



– `dask.dataframe.read_csv()`

- Doesn't load data in memory
- Infers column types
 - Samples data
 - Set data types
 - Use Parquet for data and types together

– Partitions = independent data chunks

- E.g., 33 partitions
- Graph = 99 tasks
- Each partition reads, splits data, initializes df object



9 / 16

– Consider:

- The code snippet demonstrates how to use Dask to read a CSV file.
- `dask.dataframe` is imported to handle large datasets efficiently.
- `dd.read_csv('nyc-parking-tickets-2017.csv')` reads the CSV without loading the entire file into memory, which is useful for large datasets.
- `df.isnull().sum()` calculates the number of missing values in each column.

– `dask.dataframe.read_csv()`:

- *Doesn't load data in memory:* Dask reads data in chunks, allowing for processing of large files that don't fit in memory.
- *Infers column types:* Dask samples the data to determine data types, which can be adjusted if needed.
- *Use Parquet for data and types together:* Parquet files store data and schema, making them efficient for storage and retrieval.

– **Partitions = independent data chunks:**

- Data is divided into partitions, allowing parallel processing.
- Example shows 33 partitions, resulting in 99 tasks in the computation graph.
- Each partition is processed independently, reading and splitting data, and initializing a DataFrame object.

– **Graph Visualization:**

- The diagram illustrates the task graph, showing the sequence of operations.
- Tasks like `read-csv`, `isnull`, and `aggr` are represented as nodes in the graph.

- This structure allows Dask to optimize and parallelize operations efficiently.

10 / 16: Low Level APIs: Delayed

Low Level APIs: Delayed

- Handle computations that don't fit in native Dask data structures
 - E.g., Dask DataFrame
- In the example below there is parallelism that can be exploited

```
def inc(x):
    return x + 1

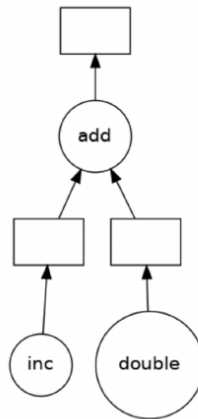
def double(x):
    return x * 2

def add(x, y):
    return x + y

data = [1, 2, 3, 4, 5]

output = []
for x in data:
    # (x + 1) + (x * 2) = 3x + 1
    a = inc(x)
    b = double(x)
    c = add(a, b)
    # 1 -> 4
    # 2 -> 7
    # 3 -> 10
    # 4 -> 13
    # 5 -> 16
    output.append(c)

# 4 + 7 + 10 + 13 + 16 = 20 + 20 + 10 = 50
total = sum(output)
print(total)
```



10 / 16

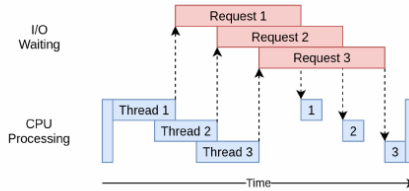
- **Low Level APIs: Delayed**
 - *Purpose:* These APIs are used to manage computations that don't naturally fit into Dask's built-in data structures like Dask DataFrame.
 - *Example:* The code snippet demonstrates how computations can be parallelized using basic Python functions.
- **Code Explanation**
 - **Functions:**
 - `inc(x)`: Increments the input `x` by 1.
 - `double(x)`: Multiplies the input `x` by 2.
 - `add(x, y)`: Adds two numbers `x` and `y`.
 - **Data:** A list `[1, 2, 3, 4, 5]` is processed.
 - **Computation:**
 - For each element `x` in `data`, `inc(x)` and `double(x)` are computed.
 - The results are added using `add(a, b)`.
 - The output is appended to a list, resulting in `[4, 7, 10, 13, 16]`.
 - **Result:** The sum of the output list is calculated, yielding 50.
- **Parallelism Potential**
 - The diagram illustrates the dependency graph of the operations.
 - *Parallelism:* The `inc` and `double` functions can be executed independently for each element, showcasing potential for parallel execution.
- **Context**
 - This example highlights how Dask's low-level APIs can be used to optimize computations

by exploiting parallelism, even when data doesn't fit into standard Dask structures.

11 / 16: Low Level APIs: Futures

Low Level APIs: Futures

- In parallel programming, a “future” encapsulates asynchronous execution, representing the eventual result
- Python `concurrent.futures`
 - High-level interface for asynchronous execution
 - Thread pool or Process pool (`Executor` interface)
- Dask extends `concurrent.futures`
 - Express everything as futures
 - Specify blocking and non-blocking



```

def inc(x):
    return x + 1

def add(x, y):
    return x + y

a = client.submit(inc, 10)
b = client.submit(inc, 20)

>>> a
<Future: status: pending, key: inc-b8aaf26b99466a7a>

>>> a
<Future: status: finished, type: int, key: inc-b8aaf26b99466a7a>

>>> a.result() # blocks until task completes and data arrives
11

```

11 / 16

- In parallel programming, a “future” encapsulates asynchronous execution, representing the eventual result
 - A *future* is a placeholder for a result that is initially unknown because the computation is still ongoing.
 - It allows programs to continue running other tasks while waiting for the result.
- Python `concurrent.futures`
 - Provides a high-level interface for executing tasks asynchronously.
 - Supports both thread-based and process-based parallelism through the `Executor` interface.
 - Useful for managing and coordinating multiple tasks that can run concurrently.
- Dask extends `concurrent.futures`
 - Dask builds on this concept by allowing everything to be expressed as futures.
 - Offers flexibility to specify whether tasks should block (wait for completion) or not, enhancing control over task execution.

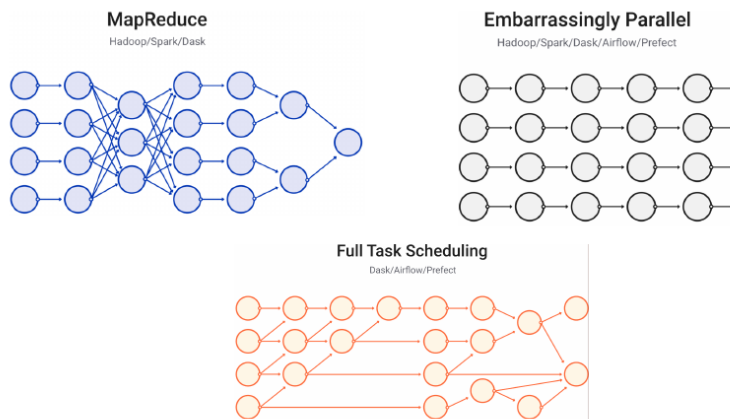
The images illustrate how futures work in practice:

- The first image shows a timeline of I/O waiting and CPU processing, highlighting how tasks can overlap in execution.
- The code snippets demonstrate submitting tasks to a client, where tasks are represented as futures.
- The status of a future can be checked, and the result can be retrieved once the task is complete, as shown in the last image.

12 / 16: Different Types of Parallel Workload

Different Types of Parallel Workload

- Break program in medium-size tasks of computation



12 / 16

- **Different Types of Parallel Workload**
 - **Break program in medium-size tasks of computation**
 - This involves dividing a program into smaller, manageable tasks that can be executed simultaneously. This approach helps in efficiently utilizing computational resources and reducing execution time.
 - **MapReduce**
 - **Hadoop/Spark/Dask**
 - MapReduce is a programming model used for processing large data sets with a distributed algorithm. It involves two main steps: *Map*, which processes and filters data, and *Reduce*, which aggregates the results.
 - Tools like Hadoop, Spark, and Dask implement this model to handle big data efficiently.
 - **Embarassingly Parallel**
 - **Hadoop/Spark/Dask/Airflow/Prefect**
 - This type of workload involves tasks that can be executed independently without requiring communication between them. It is ideal for scenarios where tasks do not depend on each other.
 - Frameworks such as Airflow and Prefect, along with Hadoop, Spark, and Dask, support this model, making it easy to scale and manage.
 - **Full Task Scheduling**
 - **Dask/Airflow/Prefect**
 - Full task scheduling involves managing complex workflows where tasks may have dependencies. This requires careful scheduling to ensure tasks are executed in the correct order.

- Dask, Airflow, and Prefect are tools that provide robust scheduling capabilities, allowing for efficient execution of intricate workflows.

13 / 16: Encoding Task Graph

Encoding Task Graph

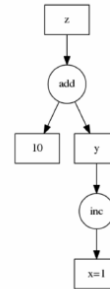
- Dask encodes tasks in terms of Python dicts and functions

```
def inc(i):
    return i + 1

def add(a, b):
    return a + b

x = 1
y = inc(x)
z = add(y, 10)
```

```
d = {'x': 1,
     'y': (inc, 'x'),
     'z': (add, 'y', 10)}
```



```
import dask.dataframe as dd

df = dd.read_csv('myfile.*.csv')
df = df + 100
df = df[df.name == 'Alice']
```

```
{
  # From the dask.dataframe.read_csv call
  ('read-csv', 0): (pandas.read_csv, 'myfile.0.csv'),
  ('read-csv', 1): (pandas.read_csv, 'myfile.1.csv'),
  ('read-csv', 2): (pandas.read_csv, 'myfile.2.csv'),
  ('read-csv', 3): (pandas.read_csv, 'myfile.3.csv'),

  # From the df + 100 call
  ('add', 0): (operator.add, ('read-csv', 0), 100),
  ('add', 1): (operator.add, ('read-csv', 1), 100),
  ('add', 2): (operator.add, ('read-csv', 2), 100),
  ('add', 3): (operator.add, ('read-csv', 3), 100),

  # From the df[df.name == 'Alice'] call
  ('filter', 0): (lambda part: part[part.name == 'Alice'], ('add', 0)),
  ('filter', 1): (lambda part: part[part.name == 'Alice'], ('add', 1)),
  ('filter', 2): (lambda part: part[part.name == 'Alice'], ('add', 2)),
  ('filter', 3): (lambda part: part[part.name == 'Alice'], ('add', 3)),
}
```

13 / 16

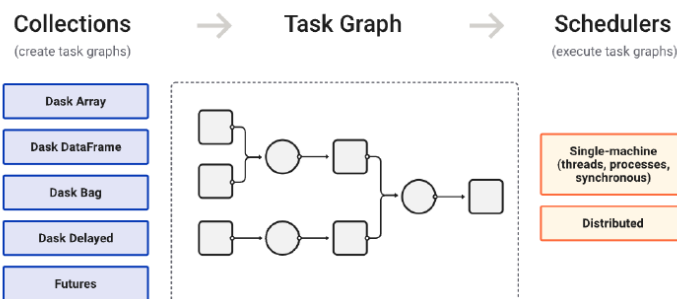
- **Encoding Task Graph**
 - Dask encodes tasks in terms of Python dicts and functions
- **Python Functions and Variables:**
 - The code snippet shows two simple functions, `inc` and `add`, which increment a number and add two numbers, respectively.
 - Variables `x`, `y`, and `z` are defined using these functions.
- **Task Graph Representation:**
 - Dask represents computations as task graphs using Python dictionaries.
 - Each key in the dictionary represents a task, and the value is a tuple containing the function and its arguments.
 - For example, `d = {'x': 1, 'y': (inc, 'x'), 'z': (add, 'y', 10)}` shows how tasks are encoded.
- **Visual Representation:**
 - The task graph is visually represented with nodes and edges, showing dependencies between tasks.
 - This helps in understanding the flow of computations and parallel execution.
- **Dask DataFrame Example:**
 - Dask can handle large datasets by breaking them into smaller chunks.
 - The code reads multiple CSV files, adds 100 to each element, and filters rows where the name is 'Alice'.
- **Task Graph for DataFrame Operations:**

- The task graph for these operations is shown as a dictionary.
- Each operation (`read-csv`, `add`, `filter`) is broken down into tasks for each file chunk.
- This allows Dask to execute operations in parallel, improving efficiency.
- **Parallel Processing:**
 - By encoding tasks this way, Dask can optimize and parallelize computations, making it suitable for big data processing.

14 / 16: Task Scheduling

Task Scheduling

- Data collections (Bags, Arrays, DataFrame) and operations create task graphs
 - Nodes: Python functions
 - Edges: Dependencies (output from one task used as input in another)
- Schedule task graphs for execution
 - Single-machine scheduler
 - Use local process or thread pool
 - Runs on a single machine
 - Distributed scheduler
 - Runs locally or across a cluster



14 / 16

- **Task Scheduling**
 - In data processing, collections like *Bags*, *Arrays*, and *DataFrames* are used to create task graphs. These graphs represent the workflow of operations.
 - **Nodes** in the graph are Python functions that perform specific tasks.
 - **Edges** represent dependencies, meaning the output of one task is used as the input for another.
- **Schedule task graphs for execution**
 - **Single-machine scheduler**
 - Utilizes local resources such as process or thread pools.
 - Designed to run on a single machine, making it suitable for smaller tasks or development purposes.
 - **Distributed scheduler**
 - Can operate locally or across a cluster of machines.
 - Ideal for handling larger datasets and more complex computations by distributing tasks across multiple machines.
- The diagram illustrates the flow from collections to task graphs and finally to schedulers, highlighting the transition from data structures to execution strategies.

15 / 16: Task Scheduling

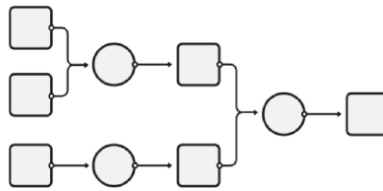
Task Scheduling

– **Dask task scheduler orchestrates work dynamically**

- Not static scheduling like a relational DB
- During computation, Dask dynamically assesses:
 - Completed tasks
 - Remaining tasks
 - Free resources (CPUs)
 - Data location

– **Dynamic approach handles various issues**

- Worker failure
 - Re-run tasks
- Workers completing at different speeds due to:
 - Different computation
 - Different hardware
 - Varying server workloads
 - Slower data access
- Network unreliability
 - Re-run or remove



15 / 16

– **Task Scheduling**– *Dask task scheduler orchestrates work dynamically*

- Unlike static scheduling found in relational databases, Dask uses a dynamic approach. This means it can adjust the scheduling of tasks on-the-fly based on current conditions.
- During computation, Dask evaluates:
 - **Completed tasks:** Keeps track of what has been finished to avoid redundant work.
 - **Remaining tasks:** Identifies what still needs to be done.
 - **Free resources (CPUs):** Allocates tasks to available processors to optimize performance.
 - **Data location:** Considers where data is stored to minimize data transfer times.

– **Dynamic approach handles various issues**– **Worker failure**

- If a worker fails, Dask can re-run tasks on other available workers, ensuring reliability.

– **Workers completing at different speeds**

- Variability in task completion can occur due to:
 - Different computational requirements for tasks.
 - Hardware differences among workers.
 - Varying workloads on servers.
 - Slower access to data.

– **Network unreliability**

- Dask can re-run tasks or remove nodes that become isolated due to network issues,

maintaining the integrity of the computation process.

16 / 16: Dask vs Spark

Dask vs Spark

- **Pros**
 - Popular framework for large datasets
 - In-memory alternative to MapReduce/Hadoop
- **Cons**
 - Java library, supports Python via PySpark API
 - Python code runs on JVM
 - Debugging is difficult as execution is outside Python
 - Different DataFrame API than Pandas
 - Learn “the Spark way”
 - May need to implement twice for exploratory analysis and production
 - Optimized for MapReduce operations
 - Difficult to set up and configure

16 / 16

- **Dask vs Spark**
- **Pros**
 - **Popular framework for large datasets:** Both Dask and Spark are widely used for handling large datasets. They are designed to process data that doesn't fit into the memory of a single machine, making them suitable for big data applications.
 - **In-memory alternative to MapReduce/Hadoop:** Unlike traditional MapReduce or Hadoop, which rely heavily on disk storage, both Dask and Spark perform computations in memory. This can lead to faster processing times as data doesn't need to be read from or written to disk as frequently.
- **Cons**
 - **Java library, supports Python via PySpark API:** Spark is primarily a Java-based library. While it supports Python through the PySpark API, this means that Python code is executed on the Java Virtual Machine (JVM).
 - **Python code runs on JVM:** This can introduce some overhead and complexity, as Python code is not running natively.
 - **Debugging is difficult as execution is outside Python:** Debugging can be challenging because the execution context is outside of the native Python environment, making it harder to trace errors.
 - **Different DataFrame API than Pandas:** Spark's DataFrame API is different from Pandas, which is a popular data manipulation library in Python.
 - **Learn “the Spark way”:** Users need to learn Spark's specific way of handling data, which can be a learning curve for those familiar with Pandas.

- **May need to implement twice for exploratory analysis and production:**
Often, data scientists use Pandas for exploratory data analysis and then need to rewrite the code in Spark for production, which can be time-consuming.
- **Optimized for MapReduce operations:** Spark is particularly optimized for MapReduce-style operations, which might not be ideal for all types of data processing tasks.
- **Difficult to set up and configure:** Setting up and configuring Spark can be complex, especially for those who are new to distributed computing frameworks. This can be a barrier to entry for smaller teams or individuals.